



Experiment – 5

Name: KONGHO SEYDOU CISSE

UID: 24BAI70013

Branch: AIML

Section/Group: 24NTPP-2A

Semester: 5th

Date of Performance:

Subject: Full Stack II

Subject Code:24CSP-337

EXPERIMENT – 5.1

1. **AIM:** To design and implement RESTful APIs using Spring Boot with proper validation, standardized responses, and scalable architecture

2. **OBJECTIVE:**

1. To understand REST API design principles
2. To implement CRUD APIs using Spring Boot
3. To enforce consistent request-response structures
4. To apply validation using Bean Validation
5. To enable secure cross-origin communication

3. **SOFTWARE REQUIREMENTS:**

- Java (JDK 17+)
- Spring Boot
- IDE (IntelliJ / Eclipse / VSCode)
- Postman / API testing tool

4. **THEORY**

REST (Representational State Transfer) is an architectural style used to design scalable and loosely coupled web services. In RESTful APIs, data is represented as resources and accessed using standard HTTP methods such as GET, POST, PUT, and DELETE.

Spring Boot simplifies REST API development through a layered architecture. The Controller Layer handles HTTP requests and responses, the Service Layer contains the business logic, and the Repository Layer manages interaction with the database.

A good REST API follows important principles such as stateless communication, a consistent URI structure, and standardized responses. Bean Validation ensures data integrity by validating incoming data using annotations such as `@NotNull`, `@NotBlank`, and `@Size`.

CORS (Cross-Origin Resource Sharing) allows a frontend and backend running on different origins to communicate securely. Standardized success and error responses also make APIs easier to integrate, maintain, and consume by frontend applications.

5. IMPLEMENTATION/PROCEDURE

- Create a Spring Boot project using Maven and configure the required dependencies.
- Create the project structure using separate Controller, Service, Repository, Model, and DTO layers.
- Create the project structure using separate Controller, Service, Repository, Model, and DTO layers.
- Create PostDTO to receive post data from the frontend or API client.
- Apply Bean Validation using `@NotBlank`, `@NotEmpty`, and `@Size` annotations.
- Create the PostRepository to perform database operations.
- Create the PostService to implement business logic for post operations.
- Create PostController using `@RestController` and define REST API endpoints.
- Implement CRUD operations using GET, POST, PUT, and DELETE HTTP methods.
- Implement a standardized ApiResponse structure for consistent API responses.
- Configure the application and database connection using application.properties.
- Create a frontend interface using HTML, CSS, and JavaScript for interacting with the REST API.
- Run the Spring Boot application using Maven Wrapper.
- Test the REST API using a web browser or Postman.
- Verify that posts can be created, retrieved, updated, and deleted successfully.

6. CODE

```
package com.example.postservice.controller;

import com.example.postservice.dto.ApiResponse;
import com.example.postservice.dto.PostDTO;
import com.example.postservice.model.Post;
import com.example.postservice.service.PostService;

import jakarta.validation.Valid;

import lombok.RequiredArgsConstructor;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/posts")
@RequiredArgsConstructor
public class PostController {

    private final PostService service;

    // GET ALL POSTS
```

```

@GetMapping
public ResponseEntity<ApiResponse<List<Post>>> getAllPosts() {

    return ResponseEntity.ok(
        new ApiResponse<>(
            "success",
            "Fetched all posts",
            service.getAllPosts()
        )
    );
}

```

```

// GET POST BY ID
@GetMapping("/{id}")
public ResponseEntity<ApiResponse<Post>> getPostById(
    @PathVariable String id) {

    return ResponseEntity.ok(
        new ApiResponse<>(
            "success",
            "Fetched post",
            service.getPostById(id)
        )
    );
}

```

```

// CREATE POST
@PostMapping
public ResponseEntity<ApiResponse<Post>> createPost(
    @Valid @RequestBody PostDTO dto) {

    Post created = service.createPost(dto);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(
            new ApiResponse<>(
                "success",
                "Post created successfully",
                created
            )
        );
}

```

```
// UPDATE POST
@PutMapping("/{id}")
public ResponseEntity<ApiResponse<Post>> updatePost(
    @PathVariable String id,
    @Valid @RequestBody PostDTO dto) {

    Post updated = service.updatePost(id, dto);

    return ResponseEntity.ok(
        new ApiResponse<>(
            "success",
            "Post updated successfully",
            updated
        )
    );
}
```

```
// DELETE POST
>DeleteMapping("/{id}")
public ResponseEntity<ApiResponse<Void>> deletePost(
    @PathVariable String id) {

    service.deletePost(id);

    return ResponseEntity.ok(
        new ApiResponse<>(
            "success",
            "Post deleted successfully",
            null
        )
    );
}
}
```

7. OUTPUT

The output of the application is a functional RESTful Post Service developed using Spring Boot. The application successfully performs CRUD operations for managing social media posts through GET, POST, PUT, and DELETE REST API endpoints. Posts can be created, retrieved, updated, and deleted successfully from the MongoDB database. Bean Validation is applied to incoming post data, and invalid requests such as empty post content or content exceeding 280 characters are rejected with appropriate validation messages. The application also provides standardized API responses containing the status, message, and data fields, ensuring consistent communication between the frontend and backend. Thus, the experiment successfully demonstrates a layered Spring Boot RESTful application with MongoDB integration, CRUD operations, Bean Validation, standardized API responses, and a frontend interface.

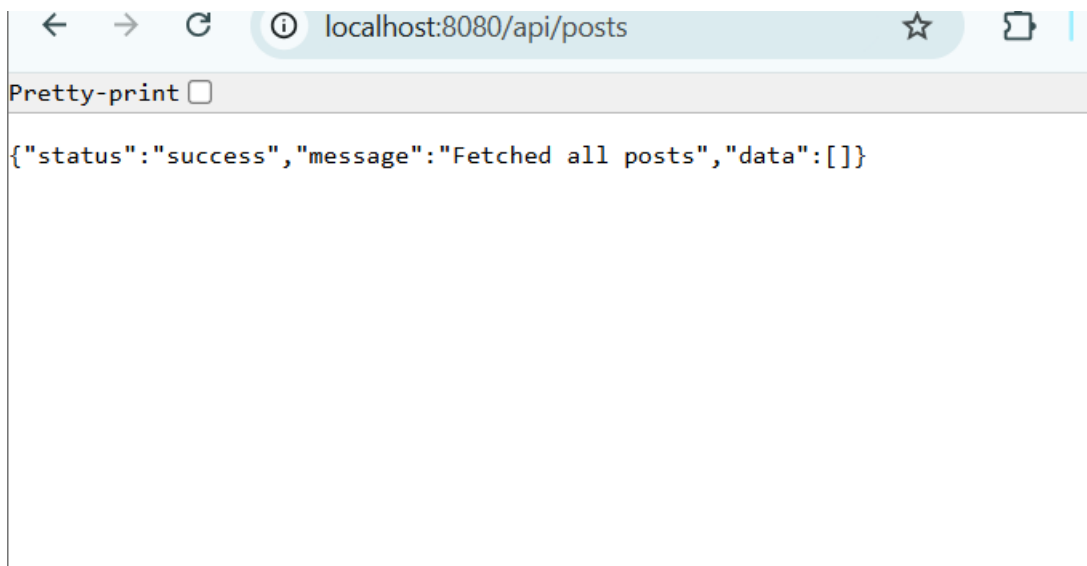


Fig 1: REST API Output – Fetch All Posts

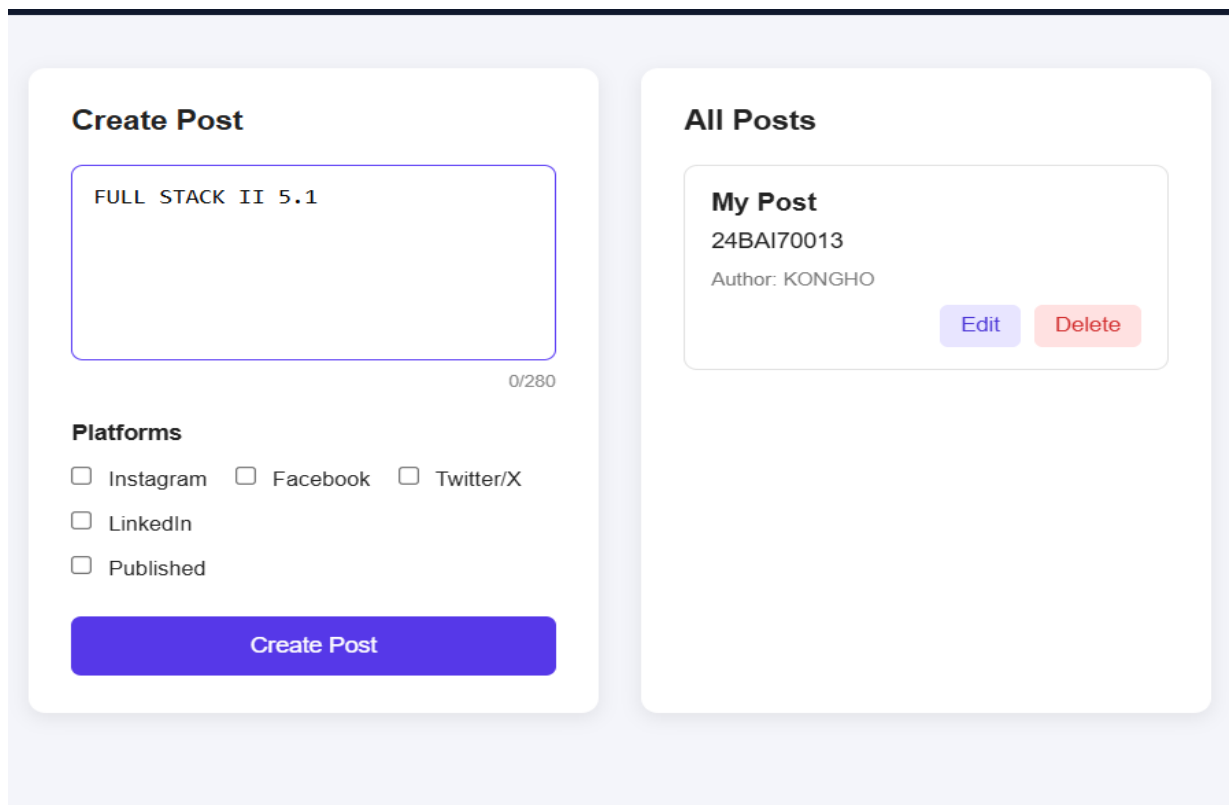


Fig 2: Post Service Frontend Interface

8. LEARNING OUTCOME

- Understand the **fundamentals of RESTful web services** and REST architecture.
- Develop REST APIs using **Spring Boot** and HTTP methods such as GET, POST, PUT, and DELETE.
- Implement a **layered architecture** using Controller, Service, and Repository layers.
- Apply **Bean Validation** to validate incoming request data.
- Configure **CORS** to enable communication between frontend and backend applications.

EXPERIMENT – 5.2

1. **AIM:** To implement global exception handling and structured logging for building robust and observable backend systems.

2. **OBJECTIVE:**

1. To handle exceptions centrally using `@ControllerAdvice`
2. To implement logging mechanisms for request tracking
3. To use correlation IDs for tracing requests
4. To improve system observability and debugging

3. **SOFTWARE REQUIREMENTS:**

- Java (JDK 17+)
- Spring Boot
- IDE (IntelliJ / Eclipse / VSCode)
- Postman / API testing tool
- Logging framework (SLF4J / Logback)

4. THEORY

In large-scale Spring Boot applications, global exception handling is used to manage errors consistently across the application. The `@ControllerAdvice` annotation allows exceptions to be handled centrally instead of writing separate exception-handling code in every controller. This improves **maintainability, consistency, and debugging**.

Logging is used to monitor application activity and identify errors. It records important information such as requests, execution time, and exceptions. Filters and Interceptors work as middleware components that can process requests and responses before or after they reach the controller. They are commonly used for **logging, authentication, and request modification**.

A Correlation ID is a unique identifier assigned to each request. Using **MDC (Mapped Diagnostic Context)**, this ID can be included in application logs, making it easier to trace a request throughout the system. This is particularly useful in distributed applications.

Together, global exception handling, logging, filters, interceptors, and correlation IDs help create a robust, maintainable, observable, and production-ready backend system.

5. IMPLEMENTATION/PROCEDURE

- Continue the PostService Spring Boot project developed in Experiment 5.1.
- Create custom exception classes such as `PostNotFoundException`.

- Create an ErrorResponse class to define a standardized error response structure.
- Create a GlobalExceptionHandler class using @ControllerAdvice.
- Implement exception handling methods using @ExceptionHandler.
- Configure logging using SLF4J and Logback.
- Add logging statements in the service layer to record important operations.
- Create a request logging filter to intercept incoming requests.
- Generate a unique correlation ID for each incoming request.
- Store the correlation ID in MDC so that it can be associated with application logs.
- Include the correlation ID in the standardized error response.
- Test a valid API request such as GET /api/posts.
- Test an invalid request such as GET /api/posts/invalid-id to generate a PostNotFoundException.
- Observe the structured error response returned to the client.
- Verify the corresponding request and error logs in the Spring Boot terminal

6. CODE

```
package com.example.postservice.exception;

import jakarta.servlet.http.HttpServletRequest;

import org.slf4j.MDC;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(PostNotFoundException.class)
    public ResponseEntity<ErrorResponse> handlePostNotFound(
        PostNotFoundException ex,
        HttpServletRequest request) {

        ErrorResponse response = new ErrorResponse(
            "error",
            ex.getMessage(),
            request.getRequestURI(),
            MDC.get("correlationId"),
            LocalDateTime.now()
        );

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
```

```

        .body(response);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneralException(
        Exception ex,
        HttpServletRequest request) {

        ErrorResponse response = new ErrorResponse(
            "error",
            "An unexpected error occurred",
            request.getRequestURI(),
            MDC.get("correlationId"),
            LocalDateTime.now()
        );

        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(response);
    }
}

```

7. OUTPUT



The screenshot shows a REST client interface with a 'Pretty-print' checkbox checked. The response is a JSON object representing a 404 error.

```

{
  "status": 404,
  "message": "Post not found with id: does-not-exist",
  "requestPath": "/api/posts/does-not-exist",
  "correlationId": "0f8c1ac2-5aec-40ff-be21-a469694ef24d",
  "timestamp": "2026-08-30T01:41:25.407400400"
}

```

Fig 1: Global Exception Handling – REST API Error Response

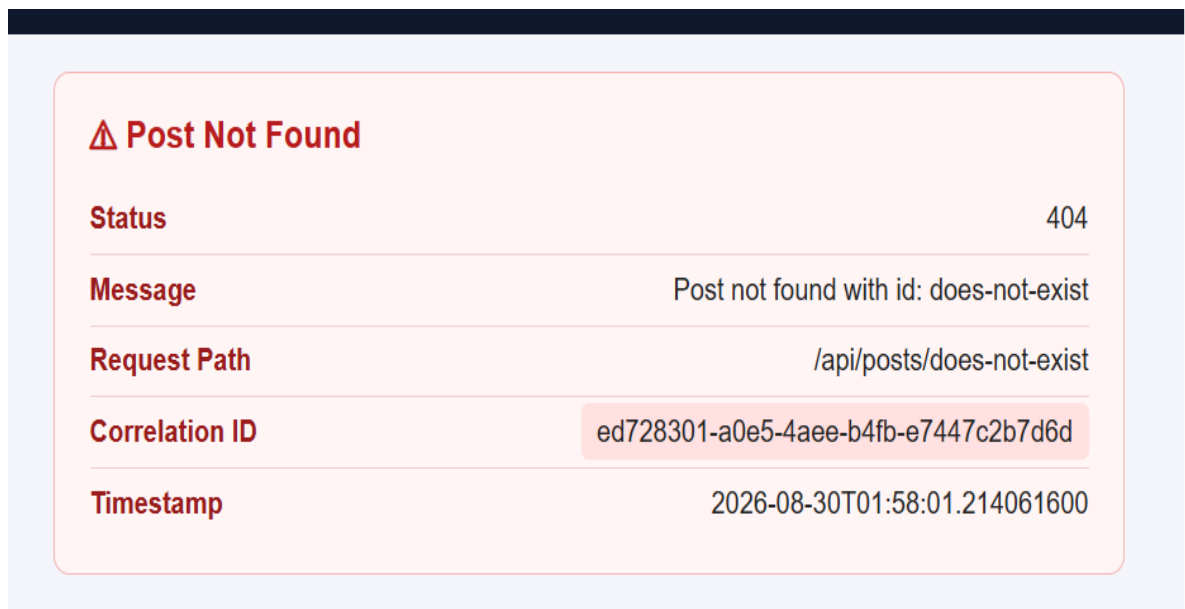


Fig 2: Frontend Global Exception Handling and Request Tracing

8. LEARNING OUTCOME

- Understand global exception handling in Spring Boot using `@ControllerAdvice`.
- Implement centralized and consistent error handling in REST APIs.
- Understand the role of logging and monitoring in backend applications.
- Use Filters and Interceptors to process HTTP requests and responses.
- Implement Correlation IDs using MDC for request tracing.